Matched signed-Frobenius rho arm for the n = 19 tau-invariant factor base: preregistration

2026-10-14

Question

The `E0 with a τ-invariant factor base` table in `../README.md` reports CPU per complete ECDLP on E0/F_2^19 (`EC1N19Ckb1h01617b86df6c`, r = 130873, ten planted scalars): 444 s for the baseline subspace base and 70 s for the τ-orbit dim-6 base (6.85×). Neither arm was compared with Pollard rho. This protocol adds a matched signed-Frobenius rho arm. The two IC arms are rerun fresh in the same job, one process, on the same scalars, interleaved (`protocol.json`, frozen before any data). The old 444 s, 70 s and 6.85× enter no statistic. Their attempt and Gröbner-call counts are compared only as a replay check of the code path.

Status: proposed / not yet measured. This commit holds the frozen protocol, the code, a pure-Python validation of the rho walk, and the workflow that runs the measurement (`.github/workflows/volcano-ic-n19-rho-arm.yml`). No CPU number in this directory is a result yet.

Design (frozen; `protocol.json`)

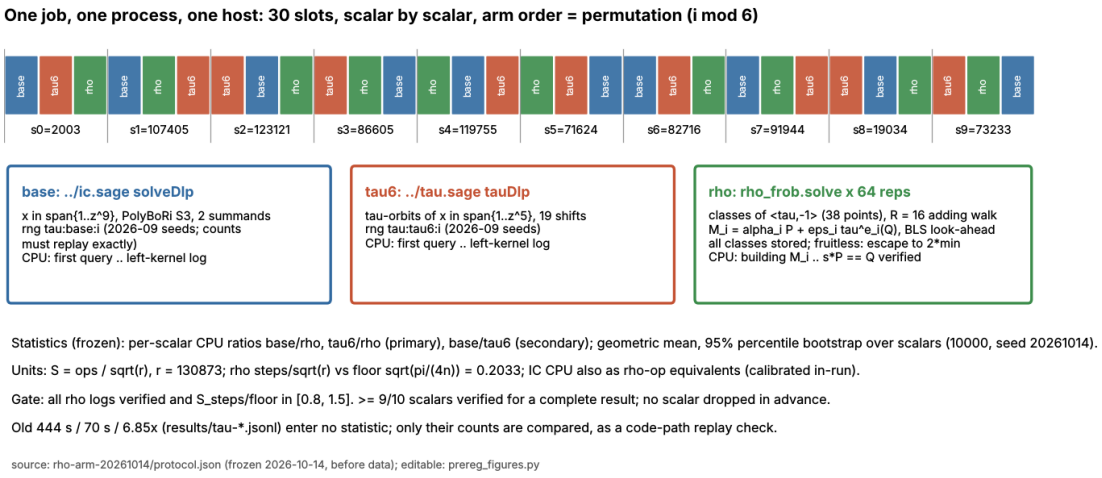


Figure 1: Interleaved schedule and arm boundaries (source: `protocol.json`; editable: `prereg_figures.py`)

- **Arms.**
  - `base` is `../ic.sage solveDlp` (x in span{1..z^9}).
  - `tau6` is `../tau.sage tauDlp` (τ-orbits of x in span{1..z^5}, 19 shifts).
  - `rho` is `rho_frob.solve` on the base arm's Sage curve object, so it uses the same Sage field and point arithmetic as the IC arms.

  - The IC arms keep their 2026-09 rng seeds, so they repeat the old work exactly; only the clock is new.
- **Rho.**
  - The walk runs on classes of $\langle \tau, -1 \rangle$, 38 points each. The canonical class member has the minimum x encoding over the 19 conjugates, then the smaller y encoding.
  - R = 16 adding walk with multipliers M_i = α_i P + ε_i τ^{e_i}(Q). The α_i P table is reusable and target-independent, so it is excluded and recorded. Each M_i costs one target-dependent addition.
  - Bernstein–Lange–Schwabe look-ahead against 2-cycles.
  - Every class is stored, i.e. distinguished points with probability 1. The walk is about 74 steps, so there is no distinguished-point tail.
  - A fruitless cycle is a repeat with identical (a, b). It is escaped deterministically to 2·min(cycle).
  - Every log is verified as s·P = Q and against the planted scalar.
  - Each scalar gets 64 independent complete single-target solves. Nothing is shared between solves.
- **Timing.**
  - `time.process_time()` of the one process, using each arm's online interval.
  - Rho's interval includes its verification. The IC arms' does not, as in ic.sage and tau.sage, which can only favour the IC arms.
- **Statistics.**
  - Primary: per-scalar CPU ratios base/rho and tau6/rho.
  - Secondary: base/tau6, and rho steps/$\sqrt{r}$ against the floor $\sqrt{(\pi/(4n))} = 0.20331$.
  - Summaries are geometric means with a 95% percentile bootstrap over scalars (10000 resamples, seed 20261014).
  - The decision rules for every case are written out in `protocol.json`.
- **Unit.**
  - S = ops/$\sqrt{r}$.
  - The IC arms have no group-operation count that covers Gröbner work. Their CPU is therefore also expressed in rho-operation equivalents, using this run's rho CPU per operation. That is a calibrated conversion, not a count.
  - Their nominal target-construction group operations are reported as a lower bound.
- **Tier.** Exploratory. A GitHub-hosted VM is neither Tier 0 nor Tier 1.

Validation of the rho implementation (measured, step counts only)

`validate_rho.py` uses pure Python, because this container has no Sage. It rebuilds E0 and the `run.sage` generator procedure. Both lifts are tried, since Sage's `lift_x` order is not reproduced; the two are P and −P. It then runs the frozen rho design on the ten planted scalars. Results are in `validation/local-validation.json`. **Labels:**

- *verified*: every log was independently checked.
- *measured*: step and operation counts.
- No timing is recorded.
- **Structure checks (verified).**
  - P has order r.
  - τ(P) = λP with $\lambda^2 + \lambda + 2 \equiv 0 \pmod{r}$.
  - On 200 random points, the canonical form is invariant over all 38 class members.
  - The coefficient unit $\varepsilon \cdot \lambda$^e maps W to its canonical form.
- **Generator P, 640 solves (10 scalars × 64), verified.**
  - 640/640 logs are correct.
  - Mean 73.4 steps, S_steps = 0.2028 (95% CI 0.1943–0.2112). That is 0.997× the floor $\sqrt{(\pi/(4n))} = 0.2033$.
  - Mean 94.2 target-dependent group operations, S_ops = 0.260.
  - 0 fruitless cycles and 0 restarts.
- **Generator −P, 80 solves, verified.** 80/80 correct; S_steps / floor = 1.04.
- **Escape path (verified).**

  - The default design produced no fruitless cycle, so the escape code was stressed separately with R = 2 and look-ahead off.
  - That run had 206 fruitless cycles in 200 solves, all escaped, and 200/200 logs were correct.

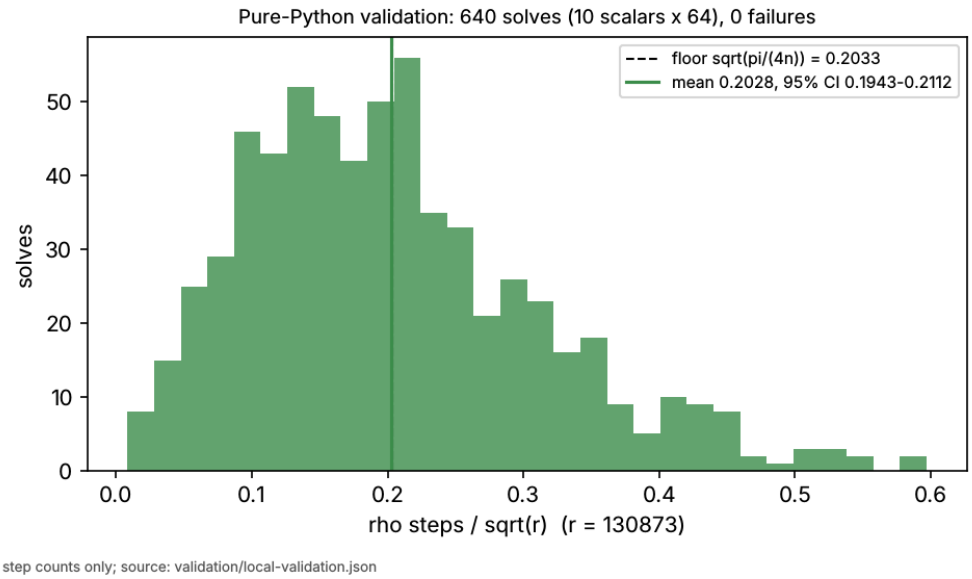


Figure 2: Rho steps/$\sqrt{\text{r}}$ of the 640 validation solves against the floor (source: `validation/local-validation.json`)

Still to check (open).

- The Sage backend has not run here. The workflow's `selftest` step asserts, before the measured run, that it agrees with the pure-Python backend on additions, doublings, negation, Frobenius and canonical forms on 200 points. It also asserts identical walks on 8 solves.
- The local passagemath attempt was abandoned without installing. The narrow wheel set (schemes, brial, repl, ntl, pari and their dependencies) is 512 MB compressed, over the disk cap in this shared container. The Sage route is therefore the pinned `sagemath/sagemath:10.10` image in GitHub Actions.

Expected runtime (proposed)

- On the 2026-09 host a scalar costs about 444 s (base) + 70 s (tau6). The 64 rho solves are well under 5 s in Sage; the walk is about 94 group operations.
- That totals about 86 min for ten scalars.
- Allowing GitHub runners to be 1.3–2.5× slower for PolyBoRi gives about 2–3.6 h. The job limit is 350 min.
- No scalar is dropped in advance. A timeout leaves the completed scalars, which are reported as incomplete without extrapolation.

What each outcome will mean (pre-stated)

- **X/rho for X in {base, tau6}.**
  - Upper CI below 1: an exploratory advance claim that needs Tier 1 replication on an independent host.
  - Lower CI above 1: X is not an advance against rho on E0/F_2^19, whatever its speedup over the baseline base.
  - Otherwise: inconclusive.
- **base/tau6.** Replicates, reverses or is inconclusive, by the same CI rule. This run's value is reported, not 6.85×.
- **Rho validity gate.** All logs must be verified and S_steps/floor must lie in [0.8, 1.5]. Otherwise the comparisons against rho are provisional.

Graphs checked

- `../figures/tau.png` and `../figures/ecdlp.png` remain current: no new measurement exists yet.
- `papers/ecdlp-constant` is unchanged; it does not cite this table.
- The run's `figures/arms.{svg,png}` and `report/report.{md,pdf}` are produced by `analyze.py` in the workflow.

Files

file	role
`protocol.json`	frozen protocol
`rho_frob.py`	rho walk, Python and Sage backends
`arms.sage`	driver: `selftest`, `run`
`analyze.py`	statistics, report, figure, PDF
`validate_rho.py`	pure-Python validation
`prereg_figures.py`	this report's figures
`../../../.github/workflows/volcano-ic-n19-rho-arms.yml`	measurement job